# Disentangling Candidate Breadth and Trajectory Supervision in Execution-Guided Program Repair

ANONYMOUS AUTHOR(S)

We present an identification framework that separates candidate breadth from trajectory supervision in execution-guided program repair, using ZPDPatch as its experimental instrument. Execution-selected breadth can otherwise masquerade as a training-method improvement. ZPDPatch mines three executable relations from user submission trajectories, trains independent QLoRA checkpoints, and selects a validation-frozen three-member portfolio. Its controller provably preserves observed-test pass rate, enforces an optional AST edit budget, and finds a repair whenever the emitted candidate set contains an eligible one. Across 997 Seen and 250 problem-held-out last-failure instances from eventually successful Python trajectories, same-draw decomposition, temperature sweeps, decoding-matched checkpoints, and an untuned-model control separate stochastic breadth from supervision. The results reject a simple heterogeneous-policy account: breadth explains most unrestricted coverage, and five-fold problem cross-fitting finds no conclusive mixed-target advantage over a selection-matched Answer pool. With sources and counts matched, Answer gains 19.3/11.2 Seen/Unseen RR points, whereas Progress repairs are 10.08/6.83 TED units smaller and retain 5.6/6.5 more buggy-token points. A scoped audit of eight educational repair studies turns this result into a reporting contract: disclose calls, report the matched one-draw estimand, and trace the complete coverage–cost curve. A current-only three-Answer deployment repairs 72.0/81.2% of Seen/Unseen inputs, while retrieval-based LSGen remains stronger without an edit limit. Hidden-test, difficulty-matching, smaller-model, and Java audits delimit rather than erase these conclusions.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation.*

Additional Key Words and Phrases: automated program repair, programming education, large language models, submission trajectories, execution-guided repair

**ACM Reference Format:**
Anonymous Author(s). 2027. Disentangling Candidate Breadth and Trajectory Supervision in Execution-Guided Program Repair. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE '27).* ACM, New York, NY, USA, 21 pages.

## 1 Introduction

Programmers iteratively write, submit, execute, and revise code. Online judges make this loop observable at scale, but their coarse verdicts do not explain whether a revision progressed or moved away from a viable solution. These submission sequences are a largely unused source of supervision for repair.

Automated program repair (APR) offers executable, code-specific feedback. The field spans search-based generate-and-validate repair, learned transformations, and test-guided validation [Gazzola et al. 2019; Le Goues et al. 2012; Monperrus 2018; Weimer et al. 2009]. Systems for programming assignments cluster peer solutions, align a user's code to a reference, or synthesize a fixed program [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. More recent systems use language models, execution diagnostics, retrieved examples, and iterative conversations [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Most condition on the latest buggy program, possibly augmented with tests or peer programs; earlier submissions by the same user are rarely first-class repair context.

The motivation for using history is straightforward but unverified. Two users may arrive at similar programs through different revisions; their histories expose attempted edits and states they demonstrably reached. In educational terms, such reachable improvements resemble the Zone of Proximal Development (ZPD) [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002]. We

ask whether repair can exploit an observed trajectory rather than immediately replace the current program with an arbitrary correct solution. ZPD is design motivation, not a measured psychological construct or learning-outcome claim.

We use ZPDPatch, illustrated in Figure 1, as an experimental instrument for mechanism separation rather than presuming trajectory superiority. It automatically converts authentic submission trajectories into three supervised repair datasets. Progress retains verdict improvements and same-verdict transitions whose per-test outcomes improve monotonically. Strict retains only transitions that improve the online-judge verdict. Answer pairs every failed submission with the trajectory's final accepted program. Three seeds per target relation produce nine independently trained QLoRA checkpoints. Validation execution selects three checkpoints; execution and AST tree edit distance (TED) control acceptance and abstention.

Our primary contribution is an empirical identification and reporting design for mechanisms that are easy to conflate: output sampling, independent training seeds, validation-selected pool breadth, relation-specific dataset construction, and serialization of prior submissions. The $A_1 \rightarrow T_1 \rightarrow S_3 \rightarrow A_3 \rightarrow A_9 \rightarrow M_9$ ladder matches each successive opportunity. Target-matched retraining finds no stable causal benefit from serializing earlier submissions, while paired-target training identifies a reproducible tradeoff: terminal Answer targets raise coverage, whereas intermediate Progress targets produce smaller, more source-preserving repairs.

This paper makes the following contributions:

- We identify candidate breadth as an underreported confound through a scoped audit of eight educational repair studies, derive a same-draw decomposition, and specify a reporting contract separating one-draw quality, execution-selected coverage, and deployed calls.
- We present a trajectory-supervision framework that constructs three finite execution-evidence relations, solves checkpoint choice as exact validation-coverage search, and provides a generator-agnostic controller contract for non-regression, candidate-relative completeness, and edit-budget compliance.
- We evaluate the full $A_1$–$T_1$–$S_3$–$A_3$–$A_9$–$M_9$ mechanism ladder with matched history and target controls, five-fold problem-cross-fitted selection, hidden tests, two model scales, two Java holdouts, and patch-locality analyses; the evidence isolates breadth as the principal coverage mechanism and intermediate targets as a reproducible locality mechanism.

## 2 Background and Related Work

### 2.1 Structure- and Reference-Based Educational Repair

Educational APR can exploit many programs written for one specification. CLARA clusters and aligns programs before extracting a repair; SARFGEN searches and aligns reference programs; Refactory normalizes refactorings before comparison [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. Embeddings, solution-space models, and visualization propagate annotations or expose recurring strategies, while multi-function and diversified methods broaden the target beyond one correction [Choi et al. 2021; Choi and Lee 2025; Glassman et al. 2015; Heo et al. 2023; Piech et al. 2015b; Rivers and Koedinger 2017; Yi et al. 2017].

LSGen is our reference-rich baseline. It filters historical repair pairs, retrieves by edit information, generates, and may retrieve again from the new state [Dai et al. 2026]. We retain its access to other users' Seen-Train solutions for the target problem. That information is unavailable on genuinely new assignments, so LSGen is not evaluated on problem-held-out Unseen tasks. This separates repository-mature repair from repair using only a learned model, statement, and tests.

## 2.2 Neural and Large-Language-Model Repair

Classical generate-and-validate APR established both the power of broad search and the risk of test-suite overfitting [Le Goues et al. 2012; Qi et al. 2014; Smith et al. 2015; Weimer et al. 2009]. Neural repair learned transformations from repository histories and context-aware translation before pretrained code models enabled zero-shot and fine-tuned repair [Jiang et al. 2023; Lutellier et al. 2020; Tufano et al. 2019; Xia and Zhang 2022; Yasunaga and Liang 2021]. RING, PyDex, and FastFixer cast educational repair as conditional generation [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2022, 2024]; retrieval and localization further constrain generation with peer programs or high-precision syntax feedback [Phung et al. 2023; Zhao et al. 2024]. These systems cover programs that do not align cleanly with a reference, but make correctness and patch scope external validation concerns.

Execution-aware methods address those concerns with signals independent of model confidence. SelfAPR learns from test diagnostics; FixEval makes execution the repair-evaluation criterion; CREF carries diagnostics through a repair conversation; counterexample-guided repair combines generation, fault localization, and MaxSAT [Haque et al. 2023; Orvalho et al. 2025; Yang et al. 2024; Ye et al. 2022]. ZPDPatch likewise executes candidates, but introduces a different unit of supervision and control. Authentic trajectories define nested improvement relations rather than synthetic corruptions or manually assigned repair roles. Members share a base model and may have correlated errors, but generation is non-adaptive: no member consumes another member's output. The unchanged program remains selectable, so a failed generation cannot force a regression.

## 2.3 Adaptive Feedback, ZPD, and Submission Histories

Programming feedback ranges from correctness reports to next-step hints and complete repairs; feedback research emphasizes timeliness, task focus, and actionable information rather than correctness alone [Hattie and Timperley 2007; Keuning et al. 2018; Narciss 2008; Shute 2008]. Tutoring research studies help seeking and assistance at a productive granularity; iSnap and LLM hint systems similarly expose next-step or multi-level support [Aleven and Koedinger 2000; Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024]. The ZPD motivates support between independent and assisted performance [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002; Schulze Bernd and Chounta 2024]. We operationalize only observed reachability: a later improving submission proves that this user reached that program state, not that showing the edit causes learning or is optimal scaffolding.

Submission sequences also support knowledge tracing, which estimates latent mastery from interaction histories [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015a]. ZPDPatch instead orders source-code states using execution evidence; it neither infers mastery nor predicts a learner response. This distinction permits fully automatic repair targets while delimiting the contribution to program behavior rather than educational outcomes.

## 2.4 Positioning of This Work

Unlike reference alignment and LSGen, ZPDPatch does not require a same-problem solution repository; unlike prior LLM and iterative repair, its targets are mined as ordered transitions from the same user's authentic submissions. Its multi-checkpoint control must also be distinguished from deep ensembles and model soups, which aggregate predictive distributions or weights [Lakshminarayanan et al. 2017; Wortsman et al. 2022]. We instead execute complete programs, measure set-union repair coverage on validation problems, and retain separate checkpoints for conditional

early stopping and structural gating. Because seed diversity alone can create complementary candidates, our evaluation gives a same-target Answer pool the identical number of trained checkpoints, exhaustive size-three choices, and test-time generation budget as the mixed-target pool.

Repeated sampling is an established inference-scaling mechanism for code: pass@$k$ rises when independent draws cover different successful programs [Chen et al. 2021]. We do not claim that effect as novel. Instead, our same-draw $T_1$–$S_3$ construction measures its exact contribution inside an APR controller, then holds the three-call budget fixed while testing whether independent checkpoints or relation-mined targets add coverage or structural locality beyond sampling alone.

*A scoped audit of candidate accounting.* Table 1 codes eight directly related neural or LLM educational-repair studies using a rule frozen before this paper's extended breadth results: record explicit candidates/iterations, whether coverage unions them, whether a matched one-draw quantity is visible, and report unmentioned settings as NR. Five deploy or primarily evaluate multiple generated opportunities. Only the educational repair benchmark reports pass@1/5/10, and CREF's AVG-5 provides an expected one-draw quantity beside TOP-5; PyDex, counterexample-guided repair, and LSGen do not isolate their extra opportunities with a fixed one-candidate control [Dai et al. 2026; Koutcheme et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. This does not invalidate those methods: adaptive iterations may carry new evidence. It establishes that candidate accounting is a live identification problem rather than a restatement of pass@$k$.

This purposive audit bounds its prevalence claim to eight studies; the contract is a normative proposal, not an APR-wide estimate.

The resulting reporting contract has four fields: expected one-draw repair rate under the same decoder, execution-selected union at every deployed $k$, the fixed selection rule, and sequential calls/latency. Our ladder supplies the first and third fields; RQ4 supplies the complete $k$–cost curve rather than a single $k = 3$ point. The machine-readable audit and inclusion rule ship with the artifact.

A direct numerical port of prior systems would either grant held-out tasks method-only evidence or remove a defining component; query budgets also differ. We instead use matched controls: iterative execution-feedback Zero-shot for adaptation, Answer for target/checkpoint diversity, and LSGen for same-problem retrieval under the common runner. Unlike adaptive PyDex, CREF, and counterexample-guided repair [Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024], our candidates do not alter later queries; this makes their set-union coverage exactly replayable.

## 3 Problem Formulation

For problem $p$, let a student's $i$-th submission be $s_i = (c_i, v_i)$, where $c_i$ is the complete source code and $v_i$ is the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \ldots, s_n]$ is chronologically ordered and terminates at the first accepted submission. Let $d_p$ contain the problem statement and resource constraints, and let $\mathcal{E}_p$ be the available test suite. Re-executing $c_i$ gives a per-test outcome vector $o_i = (o_i(e))_{e \in \mathcal{E}_p}$.

A repair example consists of problem context $d_p$, an observed sequence $h$, and a complete target program $y$. Adapter-specific rules decide which submissions remain in $h$ and which later submission supplies $y$; the two need not be adjacent in the original trajectory. Given $x = (d_p, h)$, a model generates one complete program $\hat{c}$.

We call a later state *reachable* when it occurs in the same student's observed trajectory and satisfies an adapter-specific improvement rule. Reachability is an empirical property of the data, not a guarantee that the transition is minimal, comprehensible, or caused by instructional support. Our design hypothesis is that learning from such transitions can produce repairs that preserve more of the current solution than direct answer generation.

Table 1. Scoped audit of candidate accounting. "1-draw" asks whether a matched single-opportunity quantity accompanies multi-opportunity coverage; "partial" denotes AVG-5 beside TOP-5. The artifact records sources and NR fields.

| Study | Opportunities | Union/iterate | 1-draw |
|---|---|---|---|
| PyDex/MMAPR [Zhang et al. 2022, 2024] | 10/phase | yes | no |
| EPR benchmark [Koutcheme et al. 2024] | 1/5/10 | yes | yes |
| CREF [Yang et al. 2024] | 5 | yes | partial |
| PaR [Zhao et al. 2024] | NR + oracle | oracle | no |
| FastFixer [Liu et al. 2024] | 1 reported | no | n/a |
| CEGIS repair [Orvalho et al. 2025] | 1–7 | yes | no |
| LSGen [Dai et al. 2026] | 3 | yes | no |
| REFERENT [Heo et al. 2023] | NR | NR | NR |

## 4 Approach

*Trajectory construction.* We group submissions by problem and student, sort by time, and stop at first acceptance. After indentation/comment/blank-line normalization, we remove only consecutive duplicates; non-consecutive revisits remain evidence of distinct attempts. Complete source, verdict, and order are preserved, with no maximum history or transition count. Every state is re-executed. Cache keys bind problem, normalized code, test-suite identity, timeout, and memory limit, preventing incompatible reuse. Runner failures receive an explicit most-severe outcome, and Progress construction requires a complete cache.

*Adapter-specific supervision.* We assign severity 0 to AC, 1 to WA, 2 to TLE/MLE, 3 to RE, 4 to CE, and 5 to internal failures. This operational preorder mines labels; it does not claim that every WA program is semantically closer to correctness than every RE program. For equal, non-empty test keys, $o_t \preceq_{\text{sev}} o_r$ iff no outcome in $o_t$ is more severe than its counterpart in $o_r$. Test-case progress is the Pareto condition

$$\text{TCProg}(r, t) = \mathbf{1}[o_t \preceq_{\text{sev}} o_r \wedge o_t \neq o_r]\,. \tag{1}$$

*Answer.* If $s_n$ is accepted, every preceding non-accepted submission is independently paired with $c_n$. For $\tau = [S_1, \ldots, S_9]$, the rule produces $([S_1], c_9), \ldots, ([S_8], c_9)$. Thus, Answer learns a direct failed-program-to-answer mapping and receives no multi-submission history.

*Strict.* Starting with $R = [s_1]$, we scan the trajectory and append $s_t$ only when its verdict is strictly better than the last retained submission $s_r = R[-1]$: $\text{sev}(v_t) < \text{sev}(v_r)$. If $R = [r_1, \ldots, r_m]$, we create $([r_1, \ldots, r_{k-1}], c_{r_k})$ for every $k = 2, \ldots, m$. The comparison is against the last retained state, not necessarily the immediately preceding original submission.

*Progress.* Progress uses the same retained scan but appends $s_t$ when either its verdict strictly improves or $v_t = v_r$ and $\text{TCProg}(r, t) = 1$. It therefore captures monotonic per-test progress that is hidden by an unchanged coarse verdict. Prefix-target construction is otherwise identical to Strict.

The datasets are constructed independently and may overlap. A strict transition is also a progress transition, and final accepted transitions can occur in both. The objectives nevertheless differ: Answer learns direct completion from one failed program, whereas Strict and Progress learn transitions from retained prefixes.

The order is consequential and auditable. Of the retained training labels, 89.5% of Strict and 90.7% of Progress remain valid if RE is placed below WA; 82.6% and 84.6% remain valid when every non-AC verdict is tied. This retrospective label audit does not estimate retrained-model performance, so we treat the chosen order as part of the method specification rather than a universal correctness ranking.

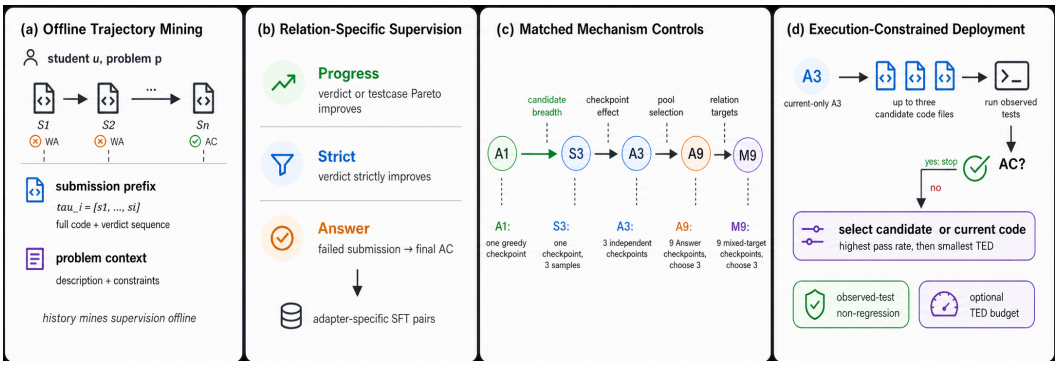


Fig. 1. Final identification and deployment design. Histories mine relation-specific SFT pairs offline. The matched $A_1 \to T_1 \to S_3 \to A_3 \to A_9 \to M_9$ ladder separates candidate breadth, checkpoint effects, pool selection, and relation targets. Recommended current-only $A_3$ generates up to three candidates; execution stops on AC and otherwise applies non-regressive, TED-aware selection.

*Execution-evidence order.* The rules form two nested evidence orders and a terminal closure. For submission $s$ with verdict $v(s)$ and outcome vector $o(s)$ over fixed tests $\mathcal{E}_p$, define $s \prec_S t$ when $\mathrm{sev}(v(t)) < \mathrm{sev}(v(s))$, and

$$s \prec_P t \iff s \prec_S t \ \vee \ (v(s) = v(t) \wedge o(t) \prec_\times o(s)), \tag{2}$$

where the strict product order means every test is no worse and at least one is better. Thus $\prec_S \subseteq \prec_P$; both are acyclic because each edge lexicographically decreases verdict severity and summed test severity. Strict retains a $\prec_S$ chain, Progress a non-compensatory $\prec_P$ chain, and Answer applies terminal closure $A(s) = s_n$ from each failed state to first AC. The guarantee concerns mined labels, not learned behavior: cache keys bind the test universe and limits, incomplete vectors fail closed, and generated programs still require execution.

*Prompt and adapter training.* All adapters share one role-neutral prompt renderer: statement and limits, followed by every submission stored in that training record in chronological order, with verdict/outcome/edit summaries, and a request for one complete solution in the dataset's language and format. The construction rule therefore determines the training context: Answer records contain only the current failed submission, whereas Strict and Progress records contain the retained prefix through that submission. No instruction names a policy or asks for a particular edit size, so specialization can arise only from the input–target distribution. Only assistant target tokens contribute to loss.

Canonical evaluation gives every checkpoint the same authentic prefix. This creates a deliberate Answer train–test context shift; RQ6 repeats selection and testing with current-only prompts. Each adapter independently minimizes masked target-token cross entropy with frozen base weights and its own QLoRA parameters.

*Validation-selected execution portfolio.* Training loss does not measure repair complementarity. For independently trained checkpoints $V = V_P \dot\cup V_S \dot\cup V_A$, let $R_v \subseteq U$ be validation instances repaired by $v$ and maximize

$$f(S) = |\bigcup_{v \in S} R_v| / |U|. \tag{3}$$

We exhaustively search all $\binom{9}{3} = 84$ triples and the 27 one-per-relation triples on one fixed-hash trajectory from each of 461 validation problems. For $\mathcal{B} = \{5, 10, 20, 40, 80, 160\}$, $R_v^B$ retains only

repairs with TED at most $B$; we maximize either $f_B$ or its mean over $\mathcal{B}$. These monotone finite-set objectives are solved exactly. A fixed Answer triple controls target heterogeneity, five-fold problem cross-fitting estimates selection behavior, and test outcomes never enter selection.

*Mechanism-identification ladder and exact breadth decomposition.* Let $r(P)$ be held-out RR; $A_1$ is one greedy Answer checkpoint, $T_1$ one uniformly chosen fixed stochastic draw, $S_3$ their same-draw union, $A_3$ a fixed three-seed greedy controller, $A_9$ Answer-9Choose3, and $M_9$ mixed-target-9Choose3. Adjacent contrasts separate decoding, candidate breadth, the aggregate checkpoint/decoder change, pool selection, and target construction. Since $S_3$ is stochastic and $A_3$ greedy, RQ4 also adds a decoding-matched three-checkpoint cell. For fixed candidates, let $Z_{ij}$ indicate that draw $j$ repairs instance $i$, and $M_i = \sum_{j=1}^{k} Z_{ij}$. The realized breadth contribution is exactly

$$\widehat{\Delta}_{\text{breadth}} = \frac{1}{N} \sum_i \left[ \mathbf{1}(M_i > 0) - \frac{M_i}{k} \right] \geq 0. \tag{4}$$

This quantity is positive exactly when success differs across draws and cannot attribute gain to temperature. Under conditional independence its expectation is $N^{-1} \sum_i [1 - (1 - p_i)^k - p_i]$, connecting the controller to ordinary pass@$k$/best-of-$n$ scaling [Chen et al. 2021]. Only $M_9 - A_9$ matches pool size, search, validation, selector, and call budget; the paired-target control separately matches training sources and counts.

*Execution-guided sequential repair.* At inference, we invoke the three validation-selected checkpoints in fine-to-broad relation order, breaking same-relation ties by their frozen member order. Each checkpoint receives the same complete observed evaluation trajectory, including Answer checkpoints whose SFT records contained only one submission, and produces one complete program. This fixes available evidence across policies without claiming that Answer was history-trained. Candidates never enter later prompts, avoiding synthetic off-distribution trajectories. Unrestricted execution stops at the first acceptance; budgeted execution continues past over-budget acceptances. The order is frozen before evaluation, and generated-feedback prompting is evaluated separately.

For a budget $B \in [0, \infty]$, define the feasible set explicitly:

$$C_B = \{c_i\} \cup \{c^{(j)} : \text{ted}^{\dagger}(c_i, c^{(j)}) < \infty \ \wedge \ \text{ted}^{\dagger}(c_i, c^{(j)}) \leq B\}. \tag{5}$$

Let $\text{Pass}(c)$ be the fraction of tests passed. We define $\text{ted}^{\dagger}(c_i, c) = 0$ for the unchanged fallback, the Python AST tree edit distance when both changed programs parse, and $+\infty$ otherwise. Unrestricted operation sets $B = \infty$ while retaining the finite-TED condition; budgeted operation uses its declared finite $B$. The scan accepts only candidates in $C_B$. If none is accepted, fallback selection is lexicographic:

$$c^* = \arg\max_{c \in C_B} \left( \text{Pass}(c), -\text{ted}^{\dagger}(c_i, c) \right). \tag{6}$$

The fallback selector maximizes executed correctness, then minimizes change; when no candidate improves $c_i$, the zero-TED fallback makes the system abstain.

*Certified selection and cost.*

Theorem 1 (Controller contract). *For any non-accepted current program, ordered list of generated candidates, and budget B, the controller (i) returns a program whose observed pass rate is at least that of $c_i$, (ii) returns a changed program only if its finite AST TED is at most B, and (iii) returns an accepted program iff at least one emitted candidate is both accepted and budget eligible. Consequently, member order can change calls and which accepted patch is returned, but not repair coverage for a fixed candidate set.*

Proof. An eligible accepted candidate terminates the scan and has pass rate one. If none exists, Equation 6 maximizes over $C_B$, which contains the zero-edit fallback, proving (i); Equation 5 proves (ii). The scan visits every candidate until an eligible acceptance, proving both directions of (iii). The final statement follows because existence in a finite set is permutation invariant. □

Thus deployed coverage is exactly the validation set-union objective $f_B$. The contract concerns observed tests and edit compliance, not hidden-test correctness or learned relational behavior.

Construction costs $O(nT)$ time and $O(n)$ storage; Answer is $O(n)$. Table 2 makes every adjacent opportunity and cost visible. In particular, $A_3$ dominates $A_9$ in unrestricted RR and cost; the nine-model pools are diagnostic and support constrained selection, not a universally cheaper default. $M_9/A_3$ latency is 13.62/13.32 seconds versus 24.40 for always-three LSGen, excluding cached current execution. Nine 155MB LoRA files occupy 1.4GB; deployment loads three.

## 5 Experimental Design

### 5.1 Research Questions

**RQ1: Utility and Frontier.** What unrestricted and edit-constrained utility does the execution-controlled portfolio provide relative to zero-shot and retrieval-based repair?

**RQ2: Trajectory Conditioning.** Does access to the full submission prefix improve repair over training and inference with only the current code?

**RQ3: Adapter Specialization.** Do Progress, Strict, and Answer exhibit different repair coverage and patch size?

**RQ4: Breadth Identification.** How much coverage comes from output sampling, independently trained checkpoints, validation pool breadth, and relation-specific dataset construction?

**RQ5: Problem Generalization.** Does ZPDPatch retain an advantage on problems completely excluded from training?

**RQ6: Robustness and Replication.** Are conclusions robust to scale, seeds, prompt and verdict choices, dependence, ordering, and the Java replications?

"Frozen" excludes final-test outcomes from training and selection. Canonical tests and named controls were selection- or repository-frozen before outcomes; normalized TED, retention, user strata, order replay, and heterogeneity are exploratory, and none was externally preregistered. A scale-run invariant found repeated current-program execution; before selection or analysis, a logged amendment rebound every member to the pre-existing cached baseline without changing training, candidates, candidate outcomes, TED, or analysis. Paired-target, extended-temperature, and $k$-curve protocols were repository-frozen before outcomes and report every cell; they are artifact-confirmatory, not externally preregistered.

RR is the primary effectiveness endpoint. The system contrast is portfolio versus Zero-shot; the mechanism contrast is selection-matched $M_9 - A_9$. For the post-review family, pooled five-fold problem-cross-fit Seen $M_9 - A_9$ is the primary mechanism estimate and its mean over the six fixed TED budgets is secondary. Scale and Java replications repeat those endpoints; prompt and verdict-order reruns are sensitivity analyses. Individual budgets and heterogeneity strata remain descriptive and do not select the conclusion.

### 5.2 Dataset Construction and Splits

We combine Python submissions, statements, metadata, and test cases for problems in Project CodeNet's Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the Python800 reference set. We remove post-acceptance submissions and

Table 2. Complete Seen mechanism ladder. Δ is the adjacent RR change in percentage points with problem-cluster 95% CI. "Val." counts candidate executions used for portfolio selection; calls reflect fixed-order early stop.

| Stage | RR | Δ [95% CI] | Train h | Val. | Calls |
|---|---|---|---|---|---|
| $A_1$ greedy Answer | 50.1 | — | 5.8 | 0 | 1.00 |
| $T_1$ stochastic Answer | 42.5 | −7.6 [−9.81, −5.41] | 5.8 | 0 | 1.00 |
| $S_3$ same-draw union | 57.6 | +15.1 [13.48,16.76] | 5.8 | 0 | 2.07 |
| $A_3$ three checkpoints | 61.7 | +4.1 [1.59,6.79] | 17.5 | 0 | 1.93 |
| $A_9$ choose three | 59.8 | −1.9 [−4.30, 0.32] | 52.6 | 4,149 | 1.93 |
| $M_9$ mixed choose three | 61.5 | +1.7 [−0.79, 4.29] | 37.1 | 4,149 | 1.98 |

consecutive normalized duplicates. Users must appear in at least three problems and each retained problem must contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests.

CodeNet identifies online-judge users and attempts, not classroom enrollment; we therefore use it as behavioral repair evidence rather than evidence about students or learning. The separately collected CodeWorkout replication below provides the CS1-course population.

Before splitting, we tokenize every possible Answer, Strict, conservative Progress, and final-evaluation prompt-target configuration. If any configuration exceeds 4,096 tokens, we exclude the entire trajectory rather than truncating code or history. This removes 3,660 trajectories and leaves 17,794. No later `max_history`, transition-count, code, or test truncation is applied.

We sort problems by eligible trajectory count and assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*. Within every Seen problem, we reserve at least one trajectory for Train, Validation, and Test, then obtain a deterministic 80:10:10 trajectory split. All 259 Unseen trajectories remain problem-held-out test data. Table 3 reports the resulting corpus. Users may occur in more than one split; RQ5 is problem-held-out, not user-held-out.

Adapter construction yields the training and validation sizes in Table 4. Final evaluation uses the last non-accepted submission before the recorded acceptance. The estimand is therefore conditional repair of the last failed state in an eventually successful trajectory, not repair of arbitrary failures; never-successful trajectories and earlier failed states are outside scope. Re-execution leaves 997 Seen instances from 328 of the 492 Seen problems and 250 Unseen instances from all 54 held-out problems. Other Seen problems still contribute Train or Validation trajectories but lack a Test trajectory satisfying both execution checks.

TIKTOC adds test cases to authentic CS1 Java CodeWorkout submissions [Duan et al. 2025]. Applying the same construction and splitting by student yields 1,125 trajectories from 224 students over 17 exercises: 786/158/181 train/validation/test trajectories from 157/34/33 disjoint students. Training has 3,470/1,110/1,386 Answer/Strict/Progress examples and validation 730/230/276. The same model, seeds, and exhaustive selection form $M_9$, $A_9$, and fixed $A_3$ before evaluation on 181 held-out students.

### 5.3 Baselines

*Zero-shot.* Despite its name, Zero-shot is an execution-aware iterative baseline: it uses the same Qwen base model and prompt as ZPDPatch without fine-tuning, and after a failure adds the verdict and number of passed tests before its next call. It receives at most three generations, matching ZPDPatch's maximum budget. Because it receives the full trajectory, the contrast measures the complete effect of learned adapters and sequential composition, not history in isolation. For the decoding-matched breadth audit, an additional *Base*×3 control samples the unadapted Qwen checkpoint three times from the identical current-only prompt and with the identical seeds, temperature,

Table 3. Dataset statistics after whole-trajectory context filtering. "Prefix" counts possible raw prefixes before adapter-specific filtering.

| Group | Role | Problems | Tests | Traj. | Sub. | Prefix | Users |
|---|---|---|---|---|---|---|---|
| Seen | Train | 492 | 9,446 | 14,027 | 54,481 | 40,454 | 4,012 |
| | Validation | | | 1,754 | 6,719 | 4,965 | 1,248 |
| | Test | | | 1,754 | 6,598 | 4,844 | 1,250 |
| Unseen | Test | 54 | 642 | 259 | 954 | 695 | 236 |

Table 4. Adapter supervision datasets. Validation examples are excluded from training.

| Adapter | Train examples | Validation examples |
|---|---|---|
| PROGRESS | 21,416 | 2,685 |
| STRICT | 16,973 | 2,116 |
| ANSWER | 40,454 | 4,965 |

top-$p$, token limit, and execution selector as Answer×3. This isolates whether fine-tuning adds coverage beyond stochastic breadth in the base model.

*Supervised Answer ensemble.* Answer-3Seed trains three conventional Answer adapters on the same 40,454 examples, model, hyperparameters, decoder, and call budget, varying only the seed. Answer-9Choose3 trains nine such adapters and receives the same exhaustive 84-triple validation search as the mixed pool. At evaluation, all members receive the same prefix. Thus Answer-9Choose3, not Zero-shot or Answer-3Seed, matches prompts, decoding, pool size, and selection opportunity.

*LSGen.* LSGen is an iterative retrieval-based solution generator [Dai et al. 2026]. We preserve repair-pair filtering, edit-vector retrieval, diff-guided generation, re-retrieval, and its minimal-change prompt. For Seen queries, five pairs come from other users' Seen-Train trajectories of the same problem. We store all three iterations and replay the same TED gate as ZPDPATCH. Unseen evaluation would expose same-problem correct programs and is therefore omitted.

All approaches use the same base LLM, public tests, 2.5-second per-test timeout, 2GB memory limit, and a maximum of three candidate generations. Final validation-selected, external-replication, and budget-controller runs cap every new candidate at 4,096 new tokens; reused control candidates are audited against the same bound.

### 5.4 Implementation

We use Qwen2.5-Coder-7B-Instruct [Hui et al. 2024] and train each adapter for one epoch with 4-bit QLoRA [Dettmers et al. 2023]. The base weights use NF4 double quantization and BF16 computation. LoRA targets the $q, k, v, o$ attention projections and gate/up/down MLP projections with rank 16, scaling 32, and dropout 0.05. Optimization uses paged 8-bit AdamW, learning rate $2 \times 10^{-4}$, cosine scheduling, 0.03 warmup ratio, and gradient checkpointing. Both canonical and CodeWorkout training use micro-batches of 2 with eight accumulation steps, for an effective batch of 16. Seeds 2027/2028/2029 repeat every target; Answer-9Choose3 adds Answer seeds 2030–2035 without changing data, hyperparameters, or greedy decoding. We select the lowest validation-loss checkpoint and run on one NVIDIA RTX 5090.

Two replications retain the same targets, selectors, and effective batch. The scale replication substitutes Qwen2.5-Coder-1.5B-Instruct and uses micro-batches of 4 with four accumulation steps. To avoid duplicating its large-vocabulary logits, the algebraically identical weighted FP32 token loss is reduced one sequence at a time with backward recomputation. The assignment-held-out CodeWorkout split uses 10/3/4 disjoint train/validation/test exercises (605/216/304 trajectories) and retrains both nine-checkpoint pools. It contains 213/150/161 students, including 151 shared train–test identities; it therefore isolates assignment transfer, whereas the student split isolates learners.

OpenAI Codex was used interactively to propose control and validation-selection designs, implement dataset builders, orchestrate experiments, develop statistical-analysis scripts, and assist manuscript drafting. The authors selected the final design and acceptance criteria, ran jobs on author-controlled infrastructure, reviewed code changes, checked reported values against saved machine-readable outputs, and every cited work was checked against a retrievable source; AI-produced text is not empirical evidence.

An isolated runner executes Python and compiles Java 21 in a network-disabled container. Outcomes are cached by program, tests, timeout, and memory settings; every policy uses the same cached current-program baseline and executes only new candidates. The Java harness reproduces all 181 recorded accepted tests.

### 5.5 Ablations

For RQ2, *Full Trajectory* uses each retained Strict/Progress prefix; *Current Code Only* removes earlier submissions from the identical examples and targets, resets the displayed position to $S_1$, and retrains. Answer is already current-only. Counts, model, and hyperparameters remain fixed.

RQ3 gives each adapter one candidate on the same 997 Seen instances. RQ4 compares seed-2027 P–S–A, generated-feedback P–S–A, Answer×3, Answer-3Seed (2027–2029), Answer-9Choose3 (2027–2035), one-per-relation selection, and unrestricted mixed selection. The two nine-member pools share 461 validation problems, all 84 triples, objectives, ties, and frozen tests. Every member sees the same authentic prefix and shares execution limits, three-call maximum, early stop, and fallback; hashes confirm distinct adapters.

Four repository-frozen controls address remaining identification choices. The *prompt control* removes prior submissions and reports both reselected and frozen-member contrasts. The *problem cross-fit* hashes Seen problems into five folds and excludes each test fold's identities during selection. The *verdict-order control* collapses all non-AC verdicts, rematerializes Progress/Strict pairs, and retrains both 7B adapters from the same base and seed. The *same-draw sampling control* draws three non-adaptive candidates from one Answer checkpoint at temperature 0.8/top-$p$ 0.95, evaluates each as a one-candidate $T_1$ replicate and their identical union as $S_3$, and then compares greedy $A_1$ and $A_3$. Endpoints, folds, selectors, budgets, and 10,000 problem-bootstrap draws were fixed before test generation; none is externally registered. To remove the remaining checkpoint/decoding confound, a second frozen audit uses the same current-only prompt, seeds 4101–4103, and stochastic decoder for three cells: three draws from Answer2027; one draw from each of Answer2027/2028/2029; and three draws from the untuned base model. It also sweeps the one-checkpoint union over temperatures 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, and 1.5 at fixed top-$p = 0.95$. At $T = .8$, seventeen additional fixed draws extend the same candidate stream to $k = 1, 3, 5, 10, 20$; we report the prefix-union RR, marginal gain, mean early-stop calls, and generation latency on both splits. No test cell selects a temperature, $k$, or portfolio.

Finally, the *paired-target control* addresses training-data quantity. We inner-join Progress and Answer examples by problem, user, current submission identifier, and exact current code, discard pairs with the same target, and serialize only that shared current code. Three Progress and

three Answer adapters then use identical source counts, seeds, hyperparameters, evaluation inputs, stochasticity, selector, and TED budgets. This estimates the complete effect of choosing an intermediate-progress versus terminal-answer target on a common source distribution; target semantics and the edit-distance distribution they induce cannot be separated without replacing the mined targets.

*Withheld-test consistency audit.* To separate candidate selection from success assessment, post-review audits partition each problem's tests by fixed SHA-256 hashes before regeneration. For Unseen (seed 2027), the observed side contains 332 tests and the hidden side 310, with at least three of each for all 54 problems. We recompute every displayed verdict, pass rate, and failure signature from observed tests only, regenerate the frozen ZPDPatch and Answer-9Choose3 members, and select candidates using only observed execution. A *joint repair* must pass both observed and untouched hidden tests; hidden outcomes never enter the prompt, generation, early stopping, tie breaking, or method selection. We repeat the procedure for all 328 Seen problems with a distinct seed (52027), where same-problem training solutions make leakage risk greatest.

### 5.6 Metrics and Statistical Analysis

For $M$ instances, let $c_m$ be the current program and $\hat{c}_m$ the selected result. $\text{Pass}(c) \in [0, 1]$ is the test pass fraction. Pass Rate (PR) averages $\text{Pass}(\hat{c}_m)$; Repair Rate (RR) averages $\mathbf{1}[\text{Pass}(\hat{c}_m) = 1]$; Improvement Rate (IR) averages $\mathbf{1}[\text{Pass}(\hat{c}_m) > \text{Pass}(c_m)]$.

For successfully repaired, parseable programs, $\text{TED}_{B\to F}$ measures AST distance from current to fixed code and $\text{TED}_{F\to O}$ from fixed code to the recorded accepted oracle, using the standard unit-cost ordered-tree edit formulation [Zhang and Shasha 1989]. Legacy fixed-policy reports use ZSS; validation-selected portfolios and exhaustive budget replay use the equivalent APTED implementation [Pawlik and Augsten 2016], cross-checked against ZSS on identical labeled ASTs. Because node-count difference lower-bounds unit-cost TED, pairs whose difference already exceeds the largest declared budget are stored only as $> 160$, which preserves every budget decision. RQ2 uses $\text{TED}_{F\to T}$, where $T$ is the identical adapter-specific recorded target shared by Full and Current configurations.

We report Wilson 95% intervals for RR and IR [Wilson 1927]. Paired RR comparisons use two-sided exact McNemar tests [McNemar 1947], with Holm correction within each planned comparison family [Holm 1979]. Because trajectories from one problem are dependent, our primary robustness analysis resamples problems as clusters 10,000 times with seed 2027 and reports intervals for paired RR, PR, and IR differences. We additionally report a problem-balanced estimand that gives every problem equal weight, with a problem bootstrap interval. CodeWorkout contrasts report both exercise- and student-cluster intervals because either identity can repeat within its held-out split. Paired TED differences use 10,000 instance bootstrap resamples on jointly repaired inputs; both procedures follow the nonparametric bootstrap principle [Efron 1979]. Unless stated otherwise, percentages are absolute percentage points.

## 6 Results

### 6.1 RQ1: Repair Effectiveness

Table 6 summarizes RQ1 and RQ5. On Seen problems, the strongest learned configuration is current-only Answer-3Seed: it repairs 72.0% Seen and 81.2% Unseen, versus 61.5% and 74.8% for the full-history mixed-target portfolio. We therefore expose it as the unrestricted deployment default rather than treating it as a secondary ablation. The mixed pool remains the test vehicle for relation-mined supervision and the small-budget frontier.

On Seen problems, the validation-selected mixed execution portfolio repairs 613 of 997 programs (61.5%), compared with 306 (30.7%) for Zero-shot. Their Wilson RR intervals are [58.4, 64.5] and [27.9, 33.6], respectively. ZPDPatch alone repairs 342 instances, whereas Zero-shot alone repairs 35 (exact McNemar $p < 10^{-63}$). More importantly for dependent trajectories, the 30.8-point paired RR gain remains under problem-cluster bootstrap [26.4, 35.1]; the equal-problem-weight gain is 26.1 points [21.2, 30.9]. PR increases by 11.07 points [9.16, 13.02] and IR by 25.2 points [20.8, 29.4] under the same clustered analysis.

The selection-matched Answer-9Choose3 baseline is the strictest target comparison: ZPDPatch differs by +1.71 Seen RR points [−0.79, 4.29] and 0.0 Unseen points [−3.70, 3.75]. Thus unrestricted results establish portfolio repair utility, but not a general coverage gain from relation-mined targets over an equally large pool of independently trained Answer checkpoints.

The legacy LSGen controller achieves the highest Seen execution correctness: 88.0% PR, 80.2% RR, and 82.4% IR. It alone repairs 267 instances, while ZPDPatch alone repairs 80 (exact McNemar $p < 10^{-23}$); the clustered RR difference is −18.8 points [−23.0,−14.5]. The tradeoff is patch size. ZPDPatch's mean $\text{TED}_{B\to F}$ is 21.05, versus 27.25 for Zero-shot and 88.27 for LSGen. Among jointly repaired parseable instances, ZPDPatch reduces TED by 8.46 relative to Zero-shot (95% CI [5.05, 12.55]) and by 43.34 relative to LSGen (95% CI [38.47, 48.44]). RQ6 replaces the legacy early-stop replay with an always-three LSGen controller before making the final budget-frontier comparison.

Table 5 operationalizes this tradeoff rather than treating TED as evidence of pedagogical quality. Under a maximum structural edit budget, ZPDPatch has substantially higher repair coverage through $B = 80$; LSGen crosses over only at the broadest reported budget. At $B = 20$, for example, the gate returns an accepted repair for 44.0% of inputs with ZPDPatch versus 13.9% with always-three LSGen, a +30.09-point problem-clustered difference [26.45, 33.82]. Across all six predeclared budgets, the mean difference is +14.68 points [12.03, 17.43]. The comparison therefore supports a constrained deployment claim: retrieval has higher unrestricted coverage, whereas the trajectory-trained portfolio covers more inputs when large structural replacement is disallowed. We treat TED only as an explicit change-cost axis: the benchmark supplies no evidence that a particular budget is an educational requirement or that a smaller patch improves learning.

Paired outcomes show complementary success rather than marginal exchanges. On Seen, ZPDPatch and Zero-shot repair 271 jointly, with 342 versus 35 unique repairs; against LSGen the corresponding counts are 533 jointly and 80 versus 267 uniquely. On Unseen, ZPDPatch and Zero-shot repair 142 jointly, with 45 versus 13 unique repairs. Thus LSGen has higher coverage, but neither Seen success set contains the other.

PR and IR confirm that the RR gain is accompanied by partial behavioral improvement. Patch distance separates repair from replacement: mean current-to-fix TED is 22.8% below Zero-shot and 76.2% below legacy LSGen on each method's successful set, while the jointly repaired paired analyses above establish the same direction without success-set confounding.

### 6.2 RQ2: Trajectory Conditioning

Table 7 shows no consistent advantage for Full Trajectory. On Seen Strict examples, Full raises PR by 0.7 points but lowers RR and IR by 0.7 and 1.3. On Seen Progress, it lowers PR, RR, and IR by 1.0, 0.4, and 0.6. On Unseen data, Full is worse for Strict but better for Progress. Target TED is also mixed: Full is closer to the recorded target for Seen Progress and Unseen Strict, but farther in the other conditions. None of the four paired RR differences is significant after Holm correction (minimum adjusted $p = 0.313$).

Each Full–Current pair holds training examples, targets, architecture, hyperparameters, and evaluation inputs fixed; only earlier submissions are removed and position leakage reset. Direction changes across conditions, clustered RR intervals include zero (Unseen Strict reaches zero at its

Table 5. Seen repair rate (%) after the same AST-TED deployment gate. ZPDPatch uses the validation-frozen budget-indexed portfolio; current-only $A_3$ uses the recommended three checkpoints; LSGen always generates three candidates. The last column is ZPDPatch minus LSGen with a 95% problem-cluster interval.

| Budget $B$ | ZPDPatch | Current $A_3$ | LSGen | ZPDPatch–LSGen [95% CI] |
|---|---|---|---|---|
| 5 | 22.5 | 16.9 | 4.8 | +17.7 [14.8, 20.6] |
| 10 | 32.4 | 28.5 | 8.0 | +24.4 [21.2, 27.7] |
| 20 | 44.0 | 40.3 | 13.9 | +30.1 [26.4, 33.8] |
| 40 | 50.9 | 51.4 | 30.4 | +20.5 [17.0, 24.0] |
| 80 | 56.0 | 61.0 | 50.8 | +5.2 [1.3, 9.3] |
| 160 | 57.9 | 66.8 | 67.6 | −9.7 [−14.2, −5.4] |

Table 6. Main repair results. PR, RR, and IR are percentages; TED is computed on successfully repaired, parseable programs.

| Split | Method | PR | RR | IR | $\text{TED}_{B\to F}$ | $\text{TED}_{F\to O}$ |
|---|---|---|---|---|---|---|
| Seen | Zero-shot | 76.3 | 30.7 | 43.7 | 27.25 | 27.67 |
| | LSGen | 88.0 | **80.2** | **82.4** | 88.27 | 83.23 |
| | Answer-3Seed (history) | 87.0 | 61.7 | 69.1 | 24.52 | 24.82 |
| | Answer-3Seed (current) | **90.2** | 72.0 | 78.6 | 37.94 | 35.51 |
| | Answer-9Choose3 | 85.9 | 59.8 | 68.0 | 25.46 | 24.01 |
| | ZPDPatch | 87.4 | 61.5 | 68.9 | **21.05** | **21.64** |
| Unseen | Zero-shot | 75.3 | 62.0 | 68.8 | 17.46 | 15.80 |
| | Answer-3Seed (history) | 84.8 | 73.2 | 79.2 | 13.38 | 13.86 |
| | Answer-3Seed (current) | **89.3** | **81.2** | **88.0** | 16.01 | 16.58 |
| | Answer-9Choose3 | 85.7 | 74.8 | 80.4 | 15.19 | 13.75 |
| | ZPDPatch | 84.9 | 74.8 | 79.6 | **11.40** | **11.96** |

Table 7. RQ2 trajectory-conditioning results. PR, RR, and IR are percentages. Current and Full use identical examples and targets.

| Split | Adapter | Input | $N$ | PR | RR | IR | $\text{TED}_{F\to T}$ |
|---|---|---|---|---|---|---|---|
| Seen | Strict | Current | 2,151 | 56.8 | 31.2 | 54.6 | 40.23 |
| | Strict | Full | 2,151 | 57.5 | 30.5 | 53.3 | 41.05 |
| | Progress | Current | 2,674 | 58.4 | 26.3 | 49.7 | 36.23 |
| | Progress | Full | 2,674 | 57.4 | 25.9 | 49.1 | 33.15 |
| Unseen | Strict | Current | 322 | 66.7 | 56.2 | 71.4 | 20.20 |
| | Strict | Full | 322 | 65.4 | 53.4 | 70.8 | 19.27 |
| | Progress | Current | 378 | 65.7 | 52.1 | 66.4 | 17.63 |
| | Progress | Full | 378 | 66.5 | 53.7 | 68.3 | 18.16 |

upper endpoint), and target TED is mixed. RQ1 can therefore establish complete-system utility while RQ2 does not support history serialization as a stable causal explanation.

We also searched directly for cases in which history should be most identifiable. A mechanically specified matched audit requires different users on the same problem, identical current verdict and pass rate, at least one earlier submission each, and normalized current-code similarity of at least 0.8. Only 6 Strict and 12 Progress Seen examples qualify (none Unseen); Full reaches 16.7/58.3% RR

versus 66.7/66.7% for Current. These tiny, post-hoc subsets cannot estimate a population effect, but they expose rather than conceal that the benchmark contains little counterfactual support for a history-sensitive claim.

The portfolio-level prompt control agrees. Removing history improves frozen mixed/Answer RR by 8.8/11.1 Seen points and 7.6/6.0 Unseen points; all intervals exclude zero. Current-only $A_3$ reaches 72.0/81.2% Seen/Unseen RR, and current-only mixed selection chooses exactly those Answer members. Thus earlier submissions are not merely unnecessary here; under the observed prompt distribution they impede the strongest deployment.

### 6.3 RQ3: Adapter Specialization

Table 8 exposes a coverage-size tradeoff. Answer has the highest PR, RR, and IR, but also the largest mean patch at TED 20.22. Progress has lower coverage but the smallest patch at TED 15.85. Strict and Progress have statistically indistinguishable RR (adjusted $p = 0.857$). Answer repairs 5.2 points more than Progress and 5.5 points more than Strict; problem-cluster intervals are [2.5, 7.9] and [2.9, 8.0], respectively.

The raw TED ordering compares different repaired subsets, so we pair joint repairs and resample problems. Progress is 1.84 TED smaller than Answer on 350 joint repairs [0.50, 3.26] and 1.67 smaller than Strict on 361 [0.53, 2.93]; Strict–Answer includes zero. Progress also retains 1.29 more buggy-token points [0.02, 2.57] and 2.89 more source-line points [1.16, 4.71] than Answer. Yet the median TED gap across 341 compact joint repairs is zero. These unmatched results identify a tail-sensitive specialization and motivate the source-matched RQ4 control; they do not establish learner benefit.

### 6.4 RQ4: Mechanism Identification

The original P–S–A controller reaches 59.5% RR, +9.4 points [7.6, 11.3] over its strongest member, but this aggregate does not identify why.

*Answer to RQ4.* Candidate breadth is the dominant unrestricted-coverage effect. Changing greedy decoding to one stochastic draw contributes -7.6 points [-9.81, -5.41], whereas execution-selecting three of those identical fixed draws contributes 15.1 points [13.48, 16.76]. Table 9 reports every frozen cell. Through $T = 1.0$, mean one-draw RR stays within 41.1–43.1% Seen and 61.1–62.5% Unseen, and three-draw union reaches 60.0/75.2%. At $T = 1.2$ it remains 58.6/76.4%, but $T = 1.5$ collapses to 15.8/35.2%. The $k$ curve rises monotonically to 74.8/84.0% at 20 draws, requiring 7.44/5.08 mean calls; the matched base-model three-draw RR is 30.4/65.6%.

At matched temperature, replacing three Answer2027 draws with one draw from each of three Answer checkpoints changes RR by -0.1 points [-2.00, 1.80] on Seen and 2.4 points [-0.41, 5.31] on Unseen; both intervals include zero. Answer SFT beats the matched base-model union by 27.2 points [22.99, 31.41] on Seen and 6.4 points [1.19, 11.34] on Unseen. Fine-tuning and breadth matter; checkpoint identity adds no conclusive third effect.

The SFT effect itself changes sharply across the problem boundary: the matched Answer-minus-base three-draw contrast is +27.2 Seen points [22.99, 31.41] but only +6.4 Unseen points [1.19, 11.34]. The 20.8-point gap is descriptive because the split difficulties are not randomized, yet it is too large to bury as a robustness detail. Exact-AST overlap rates of 0.6–1.3% rule out widespread verbatim target copying, not problem-specific adaptation or algorithmic memorization. Consequently, Unseen is the defensible estimate of transfer from Answer SFT; Seen measures deployment on repository-mature problems.

Enlarging only the Answer pool also fails to help: $A_9 - A_3 = -1.91$ [−4.30, 0.32]. Primary five-fold $M_9 - A_9$ is +2.2 [−0.19, 4.63], and the non-cross-fitted estimate is +1.71 [−0.79, 4.29]; equal-problem

Table 8. RQ3 individual-adapter results on 997 Seen instances.

| Adapter | PR | RR | IR | $\text{TED}_{B\to F}$ | $\text{TED}_{F\to O}$ |
|---|---|---|---|---|---|
| Progress | 73.4 | 44.8 | 52.0 | **15.85** | **15.89** |
| Strict | 74.8 | 44.5 | 51.2 | 17.45 | 18.38 |
| Answer | **77.5** | **50.1** | **57.5** | 20.22 | 20.61 |

estimates are sensitivity analyses. Generated-feedback repetition reaches 55.2%, and compulsory one-per-relation selection reaches 57.3%.

Target semantics produce a replicated coverage–locality tradeoff, not a coverage advantage. The paired-target control uses 7389/931 train/validation pairs and fixes current programs, counts, seeds, training, prompts, selectors, and test inputs. Table 10 shows Progress losing -19.3/-11.2 Seen/Unseen RR points; clustered intervals exclude zero, as do exact McNemar tests ($p = 4.60\times10^{-34}/6.17\times10^{-5}$). Its six-budget mean also loses -7.9 Seen points, while Unseen is unresolved. Thus intermediate targets do not improve coverage on the matched source distribution.

The same control confirms repair locality. On 278/134 joint parseable repairs, Answer patches are 10.08/6.83 TED units larger. Across 298/154 joint repairs, Progress retains 5.6/6.5 more token points and 8.9/10.4 more line points; every clustered interval excludes zero. This demonstrates a matched effect on patch scope, not learning.

The mixed-pool budget signal remains weaker. Across six TED budgets, budget-specific $M_9$ improves mean Seen RR over matched $A_9$ by 1.22 points [0.02, 2.42], but the corresponding Unseen interval includes zero, the 1.5B effect reverses sign, and the user-held-out Java interval includes zero. On 532 joint repairs, $M_9$ preserves 2.57 more buggy-token points [1.56, 3.64] and 3.33 more non-empty-line points [1.89, 4.83]. We therefore reject a general mixed-target coverage claim and retain source preservation as the supported specialization.

A post-hoc normalized audit finds positive $M_9 - A_9$ at relative limits 0.10, 0.20, and 0.40 (+2.80, +3.83, +2.90 points); three other limits include zero. It neither replaces the fixed frontier nor supports learner quality.

*Sensitivity of the null mechanism contrasts.* Using each frozen problem-cluster interval to estimate its standard error, the two-sided $\alpha = .05$, 80%-power normal-approximation MDE is 3.44 points for the primary cross-fitted $M_9 - A_9$ contrast, 2.64 at 1.5B, and 2.71 for the user-held-out Java contrast. The matched checkpoint-diversity MDE is 2.72 Seen and 4.09 Unseen points. Thus "no conclusive effect" means that effects of roughly these magnitudes were not resolved; it does not establish exact zero. The artifact recomputes these diagnostics from the frozen intervals.

### 6.5 RQ5: Problem Generalization

On 250 Unseen instances, ZPDPatch reaches 74.8% RR versus 62.0% for Zero-shot: +12.8 points [6.83, 18.75], with 45 versus 13 exclusive repairs ($p = 3.01 \times 10^{-5}$). On 122 joint parseable repairs, its patch is 6.41 TED units smaller [3.43, 9.89]. Current-only $A_3$, however, is the strongest learned configuration at 81.2% RR (Table 6).

On Unseen, Independent Policies exceeds repeated Answer-with-feedback, 73.2% versus 68.4% RR (15 versus three exclusive repairs; McNemar $p = .00754$), but matches independent Answer-3Seed: 73.2%, difference 0.0 [−3.2, 3.1], $p = 1.0$. Thus problem-held-out evidence supports checkpoint breadth, not relation heterogeneity; the decoding-matched contrast in RQ4 does not identify an additional checkpoint-diversity gain.

Table 9. Complete breadth accounting. Temperature cells are single/three-draw RR (%); $k$ cells are prefix-union RR/mean early-stop calls/amortized generation seconds. Every frozen temperature and $k$ is shown.

| $T$ | .2 | .4 | .6 | .8 | 1.0 | 1.2 | 1.5 |
|---|---|---|---|---|---|---|---|
| Seen | 43.1/48.4 | 42.7/51.9 | 43.0/54.8 | 42.5/57.6 | 41.1/60.0 | 36.2/58.6 | 7.1/15.8 |
| Unseen | 62.3/65.6 | 62.3/68.8 | 62.5/70.8 | 61.1/72.0 | 62.5/75.2 | 57.5/76.4 | 16.5/35.2 |

| $k$ | 1 | 3 | 5 | 10 | 20 |
|---|---|---|---|---|---|
| Seen | 41.0/1.00/0.13 | 57.6/2.07/0.26 | 63.0/2.89/0.37 | 68.8/4.61/0.59 | 74.8/7.44/0.96 |
| Unseen | 63.6/1.00/0.05 | 72.0/1.67/0.08 | 76.4/2.20/0.12 | 79.6/3.29/0.18 | 84.0/5.08/0.27 |

Table 10. Paired-target control. Δ is Progress minus Answer; RR and retention are percentage points. Positive Answer−Progress TED means that Progress makes the smaller joint repair. Intervals resample problems as clusters.

| Split | Progress/Answer RR | ΔRR [95% CI] | Mean ΔRR over budgets | Answer−Progress TED | Δ token retention | Δ line retention |
|---|---|---|---|---|---|---|
| Seen | 33.9/53.2 | -19.3 [-23.05, -15.64] | -7.9 [-10.53, -5.27] | 10.08 [7.53, 12.99] | 5.6 [4.26, 6.95] | 8.9 [6.64, 11.20] |
| Unseen | 65.6/76.8 | -11.2 [-17.37, -4.88] | -1.9 [-6.91, 3.22] | 6.83 [4.58, 9.39] | 6.5 [4.26, 8.95] | 10.4 [6.95, 14.25] |

The frozen unrestricted $M_9$ and selection-matched $A_9$ winners both reach 74.8% Unseen RR. Their paired difference is 0.0 points [−3.70, 3.75], so enlarging the candidate pool does not reveal a mixed-target coverage effect. Budget-indexed $M_9 - A_9$ averages +1.40 points [−1.24, 4.02] across the declared frontier; unlike the Seen contrast, this interval also includes zero. The problem-held-out evidence therefore supports portfolio coverage over zero-shot, not relation-specific coverage or a generalized constrained advantage.

*Answer to RQ5.* The 45 versus 13 discordant pairs establish ZPDPatch's paired advantage over Zero-shot without same-problem training. Because trajectory availability, not randomized difficulty, defines Unseen, its higher absolute RR does not imply that novelty helps repair.

A post-review matching audit makes that caveat testable. Without using repair outcomes, minimum-cost assignment pairs all 54 Unseen problems to distinct Seen problems on current pass rate, statement and code size, AST size, history length, and test count. For current-only $A_3$, problem-balanced RR is 81.8% on Unseen versus 86.4% on matched Seen: -4.6 points [-14.18, 5.30]. The interval is wide, but the apparent raw reversal disappears; we use only within-split paired contrasts for method claims.

*Withheld-slice consistency audit.* With all prompt feedback and online selection recomputed from the observed partition, ZPDPatch repairs 192/250 inputs (76.8%) and 189 of those same returned patches also pass every hidden test: joint RR 75.6% [69.9, 80.5] and a 98.4% [95.5, 99.5] hidden-confirmation rate. The selection-matched $A_9$ control repairs 186 observed and 184 jointly, for 73.6% joint RR [67.8, 78.7] and a 98.9% [96.2, 99.7] confirmation rate. The paired counts are 13 ZPDPatch-only versus eight $A_9$-only joint repairs (exact McNemar $p = .383$); the equal-problem joint-RR difference is +2.55 points [−0.86, 6.39]. Therefore both methods' observed repairs almost always survive untouched tests. Because both slices are hash partitions of one authored suite, they are strongly correlated and this result excludes only obvious slice overfitting; it is neither an independently designed semantic oracle nor evidence against plausible-patch overfitting.

### 6.6 RQ6: Robustness and External Replication

*Seen hidden tests and training-target overlap.* ZPDPatch/$A_9$ observed-only RR is 46.3/48.0% and joint hidden RR is 45.3/47.0%; the equal-problem contrast is -0.6 points [-3.23, 2.10]. Exact same-problem training-AC AST matches occur in 0.6/1.3% of generated selections. This rejects widespread exact copying, not memorization.

*Scale and independent-domain replication.* At 1.5B, $A_3 - A_1 = 13.4$ [11.16, 15.92], while $M_9 - A_9 = -0.8$ [-2.62, 1.07]. The four-exercise Java holdout is positive (3.9 points [2.99, 5.11]; per-exercise +2.5 to +5.5), but 151 users cross its boundary; the user-disjoint holdout is null (0.6 points [-1.20, 2.59]). Breadth replicates; relation composition does not.

*Problem-cross-fitted primary mechanism estimate.* Five fixed folds exclude test-problem identities during selection and give $M_9 - A_9 = 2.2$ points [-0.19, 4.63]; the secondary equal-problem interval includes zero. The six-budget mean remains 1.2 points [0.10, 2.33].

*Verdict-order model sensitivity.* Collapsing non-AC verdicts and retraining changes Progress RR by 2.3 points [0.20, 4.48] on Seen and -2.8 [-6.77, 1.19] on Unseen; all Strict intervals include zero. Verdict ordering has no portable benefit.

## 7 Conclusion

Candidate breadth supplies the largest stable unrestricted gain; within the evaluated Qwen2.5-Coder models and information conditions, current-only Answer-3Seed is therefore the tested default, and mixed-target pools are not justified for unrestricted deployment. More generally, method claims in execution-guided repair require a matched one-draw estimand and an explicit coverage–cost curve; our audit shows that this accounting is not yet routine. Paired-target training fixes source distribution and quantity and shows the precise role of trajectory mining: Answer wins unrestricted RR by 19.3/11.2 Seen/Unseen points, whereas Progress reduces joint-repair TED by 10.08/6.83 and preserves 5.6/6.5 more buggy-token points. Thus intermediate targets provide a reproducible locality mechanism, while their small-budget coverage signal remains unreplicated. The certified controller deploys either choice without observed-test regression or over-budget changes. Whether locality benefits users, or relation effects recur in larger user-disjoint populations, remains unmeasured.

## 8 Threats to Validity

*Construct validity.* Generated-test passing is not semantic equivalence; correlated held-out slices exclude only obvious slice overfitting. TED and retention measure change, not readability, learning, or pedagogy. *Internal validity.* $M_9 - A_9$ jointly changes target semantics, distance, and set size. The paired control fixes sources and count, but not semantics and induced distance; matched decoding controls $A_3 - S_3$. None was externally preregistered. *Selection and leakage.* Evaluation conditions on an eventually successful trajectory's last failure, likely easier than arbitrary or never-successful failures; early/middle states are unmeasured. Unseen excludes training problems [Allamanis 2019], whereas Seen permits problem and user overlap. Audits cannot eliminate memorization. Cross-fit is primary; equal-problem and normalized-TED results are sensitivities. *External validity and ethics.* CodeNet is not a classroom cohort; only CodeWorkout identifies students. Evidence covers one Python family, one smaller scale, and two Java splits. Identifiers are anonymized; learner utility remains unmeasured.

## 9 Data and Artifact Availability

The anonymized replication package supplies environments, a hash-bound manifest, and an `ARTIFACT.md` claim–evidence map.

## References


Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.

Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153. doi:10.1145/3359591.3359735

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.

Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The "Grey Area": A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.

Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes. Cambridge, MA: Harvard University Press* (1978).

Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.

Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Zhangqi Duan, Nigel Fernandez, Alexander Hicks, and Andrew Lan. 2025. Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference*. 235–250. doi:10.1145/3706468.3706500

Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering* 45, 1 (2019), 34–67. doi:10.1109/TSE.2017.2755013

Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.

Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction* 22, 2, Article 7 (2015), 35 pages. doi:10.1145/2699751

Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. Fixeval: Execution-based evaluation of program fixes for programming problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 11–18.

John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

Hasnain Heickal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).

Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.

Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. doi:10.2307/4615733

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE, 388–398.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*. 1430–1442. doi:10.1109/ICSE48619.2023.00125

Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.

Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.

Charles Koutcheme, Nicola Dainese, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2024. Benchmarking Educational Program Repair. *arXiv preprint arXiv:2405.05347* (2024). doi:10.48550/arXiv.2405.05347

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles. In *Advances in Neural Information Processing Systems*, Vol. 30.

Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104

Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–114. doi:10.1145/3395363.3397369

Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1, Article 17 (2018), 24 pages. doi:10.1145/3105906

Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.

Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.

Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. *Information Systems* 56 (2016), 157–173. doi:10.1016/j.is.2015.08.004

Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).

Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015a. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).

Chris Piech, Jonathan Huang, Andy Nguyen, Mike Phulsuksombati, Mehran Sahami, and Leonidas Guibas. 2015b. Learning Program Embeddings to Propagate Feedback on Student Code. In *Proceedings of the 32nd International Conference on Machine Learning*. 1093–1102.

Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.

Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladmir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).

Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziying Dai, and Chengsong Wang. 2014. The Strength of Random Search on Automated Program Repair. In *Proceedings of the 36th International Conference on Software Engineering*. 254–265. doi:10.1145/2568225.2568254

Kelly Rivers and Kenneth R. Koedinger. 2017. Data-Driven Hint Generation in Vast Solution Spaces: A Self-Improving Python Programming Tutor. *International Journal of Artificial Intelligence in Education* 27, 1 (2017), 37–64. doi:10.1007/s40593-015-0070-z

Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.

Nadine Schulze Bernd and Irene-Angelica Chounta. 2024. Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure. In *International Conference on Artificial Intelligence in Education*. Springer, 365–378.

Valerie J. Shute. 2008. Focus on Formative Feedback. *Review of Educational Research* 78, 1 (2008), 153–189. doi:10.3102/0034654307313795

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 532–543. doi:10.1145/2786805.2786825

Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology* 28, 4, Article 19 (2019), 29 pages. doi:10.1145/3340544

Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically Finding Patches Using Genetic Programming. In *Proceedings of the 31st International Conference on Software Engineering*. 364–374. doi:10.1109/ICSE.2009.5070536

Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 23965–23998.

Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971. doi:10.1145/3540250.3549101

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.

Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International conference on machine learning*. PMLR, 11941–11952.

He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. doi:10.1137/0218082

Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024). doi:10.48550/arXiv.2404.01754